STARTUB · DIA 1

Frontend Moderno com React e TypeScript

Dia 1 · Da tela à API: componentes, rotas e dados de verdade

Evento	StartUB
Encontro	Dia 1
Minicurso	Desenvolvimento Frontend Moderno com React e TypeScript
Professor	Marcos David Souza Ramos
E-mail	marcos.david@prof.unibalsas.edu.br
Carga horária	3 horas
Pré-requisitos	Material prévio concluído: Node.js, projeto criado e rodando

Centro Universitário Unibalsas · Balsas-MA
Material de apoio do minicurso

Sumário

- Como usar este material
- Roteiro do encontro
- 1. Frontend moderno em uma página
- 2. O projeto por dentro
- 3. Componentes, props e interfaces
- 4. Rotas
- 5. Cadastro e login
- 6. Falando com a API
- 7. Fechamento
- Para praticar em casa
- Referência rápida

Como usar este material

Este é um tutorial: você constrói a aplicação seguindo os **Passos**, na ordem. Cada passo mostra o arquivo que muda, com o caminho dele no topo do bloco, e o código **inteiro** daquele arquivo naquele momento. Quando um arquivo muda de novo mais adiante, ele aparece outra vez, completo. Basta substituir o conteúdo.

> **Conceito.** A ideia por trás do que você está fazendo.

> **Na prática.** Como aquilo aparece no trabalho de quem faz frontend.

> **Atenção.** O erro mais comum naquele ponto, com o sintoma que ele produz.

> **CONFIRA ANTES DE SEGUIR.** O que precisa estar funcionando antes do próximo passo.

As requisições à API aparecem como uma barra com o método, o endereço e o status, seguida da resposta, copiada de uma execução real:

```
GET  https://startub.appmake.com.br/api/ping   200 OK
```

O servidor de desenvolvimento (`npm run dev`) fica ligado o encontro inteiro, num terminal só dele. A cada arquivo salvo, a página se atualiza sozinha.

O projeto ao fim de cada etapa está no repositório do minicurso, em https://github.com/prof-marcosdavid/startub-eventos, marcado com uma tag: `d1-etapa-3` , `d1-etapa-4` , e assim por diante. Se você se perder no meio de uma etapa, comece uma branch sua a partir do fim da etapa anterior e siga com a turma:

```
TERMINAL
> git switch -c minha-versao d1-etapa-4
> npm install
```

Roteiro do encontro

	Etapa	O que você leva dela
1	**Frontend moderno em uma página**	o desenho da aplicação e de quem conversa com quem
2	**O projeto por dentro**	as pastas, o caminho do `index.html` até o `App.tsx` , o Tailwind
3	**Componentes, props e interfaces**	a vitrine de eventos com dados de exemplo
4	**Rotas**	navegação entre a vitrine e a página de cada evento
5	**Cadastro e login**	o token, o primeiro Context, formulários com `useState` e a rota protegida

	Etapa	O que você leva dela
6	**Falando com a API**	a vitrine e o detalhe com os dados reais, com carregando e erro
7	**Fechamento**	o ponto de partida do segundo encontro

1. Frontend moderno em uma página

Até pouco tempo atrás, cada clique num site pedia ao servidor uma página HTML nova, inteira. Uma aplicação moderna funciona diferente: o navegador baixa a aplicação **uma vez** e, a partir daí, troca só os **dados** com o servidor, em JSON. Quem monta as telas é o próprio navegador.

> **Conceito.** Isso se chama **SPA** (*single page application*, aplicação de página única). O servidor deixa de mandar telas e passa a oferecer uma **API**: um conjunto de endereços que recebem e devolvem dados. A mesma API pode atender o site, um aplicativo de celular e outros sistemas.

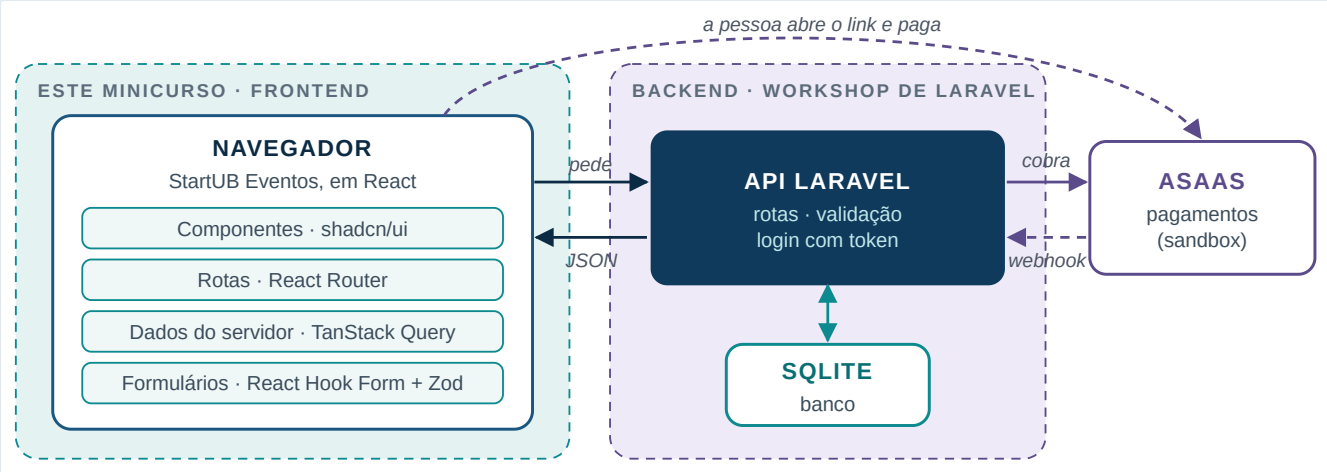


Figura 1. A aplicação que construímos e a API com que ela conversa

A Figura 1 mostra as peças. Neste minicurso construímos a parte da esquerda. A API do centro é a que a turma do workshop de Laravel constrói no StartUB, e ela também se comunica com o Asaas, uma plataforma de pagamentos, quando o evento é pago. Ao fim dos dois encontros, o StartUB Eventos terá as telas da Tabela 1.

Tela	Endereço	O que faz
Vitrine	`/`	lista os eventos, com busca e filtro
Detalhe	`/eventos/2`	mostra um evento e o formulário de inscrição
Comprovante	`/inscricoes/7`	mostra a inscrição e acompanha o pagamento
Minhas inscrições	`/minhas`	lista as suas inscrições feitas neste navegador
Painel	`/painel`	cria, edita e remove eventos

Tela	Endereço	O que faz
Criar conta	`/cadastro`	cadastro de uma conta nova
Entrar	`/entrar`	login

A API do StartUB só atende quem tem conta, então só as duas últimas telas são abertas: todas as outras exigem login. Hoje fazemos a vitrine, o detalhe, o cadastro e o login. O segundo encontro cuida do resto.

2. O projeto por dentro

Abra o projeto `startub-eventos` no VS Code e ligue o servidor, se ainda não estiver ligado.

```
TERMINAL
> npm run dev
```

2.1 As pastas

Caminho	O que guarda
`index.html`	a única página HTML da aplicação, com uma `div` vazia de id `root`
`src/main.tsx`	o ponto de partida: coloca o React dentro daquela `div`
`src/App.tsx`	o primeiro componente, a raiz da aplicação
`src/index.css`	o Tailwind e o tema: cores, fontes e efeitos do StartUB
`public/`	arquivos servidos como estão, como a logo do congresso
`src/components/ui/`	os componentes do shadcn (botão, cartão, campo), agora código seu
`src/lib/utils.ts`	a função `cn()`, que junta classes do Tailwind
`package.json`	as bibliotecas instaladas e os comandos (`dev` , `build` , `lint`)
`vite.config.ts`	a configuração do Vite, inclusive o atalho `@`

> **Na prática.** O atalho `@` aponta para a pasta `src`. Em vez de escrever `import { Button } from "../../components/ui/button"`, que muda conforme o arquivo onde você está, escreve-se sempre `import { Button } from "@/components/ui/button"`.

2.2 Tailwind em cinco minutos

O Tailwind troca o arquivo de CSS por **classes pequenas**, cada uma com uma regra, aplicadas direto no elemento. As que mais vão aparecer:

Classe	Equivale a
`p-4`, `px-4`, `py-8`	`padding` em todos os lados, nos lados, em cima e embaixo (cada unidade vale 0,25rem)
`mx-auto`, `max-w-5xl`	centralizar e limitar a largura
`flex`, `grid`, `gap-4`	flexbox, grid e o espaço entre os itens
`grid-cols-[4.5rem_1fr]`	uma grade com uma coluna de 4,5rem e outra que ocupa o resto
`sm:grid-cols-[5rem_1fr_auto]`	a partir de telas pequenas, três colunas; sem o `sm:`, vale sempre
`text-4xl`, `font-extrabold`	tamanho e peso da fonte
`font-heading`	a fonte dos títulos, a Baloo 2
`text-muted-foreground`, `bg-muted`	cores do **tema**, que mudam sozinhas no modo escuro
`rounded-xl`, `border`	cantos arredondados e borda

2.3 A identidade visual do StartUB

Uma aplicação de frontend não é só funcional: ela fala a língua visual de quem a usa. O StartUB Eventos usa a identidade do congresso, a mesma das peças de divulgação: o azul-marinho e o magenta da logo, o gradiente do violeta ao magenta nos números grandes, a letra arredondada do "2026" e as ondas na base das peças.

No material prévio, você descompactou o **kit visual do StartUB** no projeto. Ele trocou o `src/index.css` inteiro, colocou a logo em `public/startub-logo.jpg` e substituiu três componentes do shadcn, o botão, o campo de texto e a área de texto, por versões mais altas: o padrão do shadcn tem 32 pixels, pouco para um formulário de inscrição usado no celular. Como o código dos componentes é seu, ajustar o tamanho é editar as classes do `button.tsx` e do `input.tsx`. O arquivo é longo, porque repete o mapeamento de cores que o shadcn cria, mas só quatro trechos são nossos. Leia cada um no seu projeto.

PASSO 2.1 · LER O TEMA DO STARTUB

As fontes, instaladas como pacotes do npm, e dois **utilitários próprios**, criados com `@utility`: o texto em gradiente e as ondas do rodapé.

`src/index.css (trecho)`

```
@import "tailwindcss";
@import "tw-animate-css";
@import "shadcn/tailwind.css";
@import "@fontsource-variable/baloo-2";
```

```
@import "@fontsource-variable/atkinson-hyperlegible-next";

@custom-variant dark (&:is(.dark *));

/* O gradiente das peças do StartUB, do violeta ao magenta, aplicado ao texto.
   Usado nos números grandes das datas. */
@utility texto-marca {
    background-image: linear-gradient(135deg, #8b3fd6, var(--marca));
    background-clip: text;
    color: transparent;
}

/* As ondas da base das peças do congresso: magenta e violeta por cima do azul-marinho.
   A primeira camada é o desenho das ondas; a segunda pinta o resto do rodapé. */
@utility onda-startub {
    background:
        url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 1440
80' preserveAspectRatio='none'%3E%3Cpath d='M0 34C240 74 430 36 720 28S1200 80 1440
30V80H0Z' fill='%23de32bd'/%3E%3Cpath d='M0 46C240 82 420 40 720 34S1200 78 1440 42V80H0Z'
fill='%238b3fd6'/%3E%3Cpath d='M0 56C240 86 430 48 720 42S1200 84 1440 52V80H0Z'
fill='%231d2559'/%3E%3C/svg%3E") top / 100% 4.5rem no-repeat,
        linear-gradient(#1d2559, #1d2559) 0 4.4rem / 100% 100% no-repeat;
    padding-top: 5.5rem;
    color: #ffffff;
}
```

O papel de cada fonte, dentro do `@theme`, e a cor da marca virando a classe `text-marca`:

src/index.css (trecho)

```
@theme inline {
    --font-heading: 'Baloo 2 Variable', sans-serif;
    --font-sans: 'Atkinson Hyperlegible Next Variable', sans-serif;
    --color-marca: var(--marca);
```

As cores do tema claro. Repare que os nomes são **papéis** (`--primary`, `--muted`, `--border`), e não cores: os componentes do shadcn usam esses nomes, e trocar um valor aqui repinta a aplicação inteira:

src/index.css (trecho)

```
:root {
    /* A identidade do StartUB 2026, tirada da logo do congresso */
    --marca: #de32bd;              /* magenta da logo: só nos números grandes das datas */
    --background: #ffffff;
    --foreground: #1d2559;         /* azul-marinho da logo: textos e títulos */
    --card: #ffffff;
    --card-foreground: #1d2559;
    --popover: #ffffff;
    --popover-foreground: #1d2559;
    --primary: #c0249f;            /* magenta mais escuro: botões e links, com contraste
para o branco */
    --primary-foreground: #ffffff;
    --secondary: #f5f3fa;
    --secondary-foreground: #1d2559;
```

```
    --muted: #f5f3fa;
    --muted-foreground: #5a5f80;
    --accent: #f8e9f5;
    --accent-foreground: #1d2559;
    --destructive: #c62828;
    --border: #e4e1ee;
    --input: #d9d5e6;
    --ring: #c0249f;
    --radius: 0.75rem;
```

E os títulos, que passam a usar a fonte arredondada sem precisar de classe em cada um:

src/index.css (trecho)

```
  /* títulos na fonte arredondada da logo */
  h1, h2, h3 {
    @apply font-heading tracking-tight;
    }
```

Conceito. Variáveis com nome de papel, como `--primary` e `--muted-foreground`, são chamadas de **design tokens**. Elas separam a decisão visual ("o botão principal é magenta") do código dos componentes (`bg-primary`). É assim que uma equipe de design entrega uma identidade e o frontend a aplica num lugar só. O bloco `.dark`, logo abaixo no arquivo, redefine os mesmos papéis para o tema escuro.

Na prática. O magenta da logo, `#de32bd`, com texto branco por cima, não tem contraste suficiente para leitura confortável. Por isso o tema guarda dois magentas: o da logo, em `--marca`, só para os números grandes, e um mais escuro, `#c0249f`, em `--primary`, para botões e links. Abra a página, aperte `D` e veja o tema escuro.

3. Componentes, props e interfaces

3.1 Componente é uma função

Um **componente** é uma função que devolve a descrição de um pedaço da tela, escrita em **JSX**: algo que parece HTML, mas está dentro do TypeScript. Por isso os arquivos terminam em `.tsx`.

Em HTML	Em JSX
`class="titulo"`	`className="titulo"`
texto fixo	`{evento.titulo}` : qualquer expressão entre chaves
`<img src="x.png">`	`<img src="x.png" />` : toda tag precisa fechar
repetir à mão	`{lista.map(...)}` : gerar a partir de uma lista
mostrar ou esconder com CSS	`{condicao && <p>...</p>}` : só desenha se a condição for verdadeira

> **Atenção.** O nome do componente começa com **letra maiúscula**: `EventoCard`, não `eventoCard`. Com minúscula, o React acha que é uma tag HTML desconhecida e não desenha nada, sem dar erro.

3.2 O formato do dado vem da API

Antes de desenhar um evento, é preciso saber como ele chega. Esta é a resposta da API para o evento 2, consultada com o token de uma conta. O token é assunto da Etapa 5; por enquanto, basta olhar o formato:

`GET` `https://startub.appmake.com.br/api/eventos/2` **200 OK**

RESPOSTA

```
{
  "id": 2,
  "titulo": "Minicurso de Git e GitHub",
  "descricao": "Versionamento de código do zero ao pull request.",
  "local": "Auditório",
  "data": "2026-12-12 14:00",
  "vagas": 2,
  "valor": "25.00",
  "created_at": "2026-11-12T16:01:24.000000Z",
  "updated_at": "2026-11-12T16:01:24.000000Z",
  "user_id": null
}
```

O TypeScript permite descrever esse formato numa **interface**. A partir daí, o editor completa os nomes dos campos e acusa erro de digitação antes de rodar.

PASSO 3.1 · OS TIPOS

Crie o arquivo `src/tipos.ts`:

src/tipos.ts

```ts
// O formato dos dados, copiado das respostas reais da API.

export interface Evento {
  id: number
  titulo: string
  descricao: string | null
  local: string
  data: string // "2026-12-05 19:00", sem o T
  vagas: number
  valor: string // "25.00", texto e não número
  created_at: string
  updated_at: string
  user_id: number | null // quem criou; null nos eventos de exemplo da API
}
```

O último campo, `user_id`, diz qual conta criou o evento. Os eventos do StartUB vêm com `null`, porque não foram criados por ninguém pela aplicação. Ele vai importar no painel do segundo encontro.

> **Atenção.** Repare nos comentários de `data` e `valor`. O `valor` chega como **texto** (`"25.00"`), e não como número: somar `valor + 10` daria `"25.0010"`. A `data` chega como `"2026-12-12 14:00"`, com espaço no lugar do `T` do padrão ISO, e `new Date()` não entende esse formato em todos os navegadores. A interface descreve o dado **como ele é**, e quem converte é uma função própria.

PASSO 3.2 · FUNÇÕES DE FORMATAÇÃO

Crie `src/lib/formatar.ts`:

src/lib/formatar.ts

```ts
// A API manda a data como "2026-12-05 19:00". Trocar o espaço por T deixa o texto
// no formato ISO, que todo navegador entende.
export function paraDate(texto: string): Date {
  return new Date(texto.replace(" ", "T"))
}

const doisDigitos = (n: number) => String(n).padStart(2, "0")

// As partes da data que a tela usa separadas: "05", "dez", "sábado", "19h00".
export function partesDaData(texto: string) {
  const data = paraDate(texto)
  return {
    dia: doisDigitos(data.getDate()),
    mes: data.toLocaleDateString("pt-BR", { month: "short" }).replace(".", ""),
    semana: data.toLocaleDateString("pt-BR", { weekday: "long" }),
    hora: `${doisDigitos(data.getHours())}h${doisDigitos(data.getMinutes())}`,
  }
}

// "sábado, 05 de dez, 19h00"
export function formatarData(texto: string): string {
  const { dia, mes, semana, hora } = partesDaData(texto)
  return `${semana}, ${dia} de ${mes}, ${hora}`
}

const formatoMoeda = new Intl.NumberFormat("pt-BR", {
  style: "currency",
  currency: "BRL",
})

// O valor chega como texto ("25.00"). Number() converte antes de formatar.
export function formatarValor(valor: string): string {
  const numero = Number(valor)
  return numero === 0 ? "Gratuito" : formatoMoeda.format(numero)
}

export function ehGratuito(valor: string): boolean {
  return Number(valor) === 0
}
```

O `Intl` e o `toLocaleDateString`, embutidos no navegador, falam português sem biblioteca extra. A função `partesDaData` devolve as partes da data separadas (`"05"`, `"dez"`, `"sábado"`, `"19h00"`), porque a tela desenha

cada uma de um jeito. Os dois cuidados do Passo 3.1 ficam resolvidos aqui, num lugar só.

3.3 Props: o que o componente recebe

Um componente recebe dados de quem o usa por meio de **props**, como os atributos de uma tag. A interface das props diz o que ele aceita.

PASSO 3.3 · A DATA DO EVENTO

A peça mais marcante da tela é a data: o dia em número grande, no gradiente do StartUB, como o "09 a 13" das peças do congresso. Ela aparece na programação, na página do evento e no comprovante, então vira um componente. Crie `src/components/DataDoEvento.tsx`:

`src/components/DataDoEvento.tsx`

```
import { partesDaData } from "@/lib/formatar"
import { cn } from "@/lib/utils"

interface DataDoEventoProps {
  data: string
  className?: string
}

// O dia em número grande, no gradiente das peças do StartUB, com o mês e o dia da semana
embaixo.
export function DataDoEvento({ data, className }: DataDoEventoProps) {
  const { dia, mes, semana } = partesDaData(data)

  return (
    <time dateTime={data.replace(" ", "T")} className={cn("flex flex-col items-center
leading-none", className)}>
      <span className="texto-marca font-heading text-5xl font-extrabold">{dia}</span>
      <span className="mt-1 font-semibold">{mes}</span>
      <span className="mt-0.5 text-xs opacity-70">{semana}</span>
    </time>
  )
}
```

Duas props: a `data`, obrigatória, e a `className`, opcional, marcada com `?`. A função `cn()` junta as classes do próprio componente com as que vierem de fora, para quem usa poder ajustar o espaçamento. O `<time dateTime>` diz ao navegador e aos leitores de tela que aquilo é uma data.

PASSO 3.4 · O CARTÃO DE UM EVENTO

Crie `src/components/EventoCard.tsx`:

`src/components/EventoCard.tsx`

```
import { Clock, MapPin, Users } from "lucide-react"
import { DataDoEvento } from "@/components/DataDoEvento"
import { Badge } from "@/components/ui/badge"
import { Card } from "@/components/ui/card"
import { ehGratuito, formatarValor, partesDaData } from "@/lib/formatar"
```

```tsx
import type { Evento } from "@/tipos"

interface EventoCardProps {
  evento: Evento
}

// Uma linha da programação: a data à esquerda, o evento no meio, o valor à direita.
export function EventoCard({ evento }: EventoCardProps) {
  const { hora } = partesDaData(evento.data)

  return (
    <Card className="grid grid-cols-[4.5rem_1fr] gap-4 px-4 text-base sm:grid-cols-
[5rem_1fr_auto] sm:items-center sm:px-6">
      <DataDoEvento data={evento.data} />

      <div className="grid gap-1.5">
        <h2 className="text-xl leading-tight font-bold">{evento.titulo}</h2>
        {evento.descricao && <p className="text-muted-foreground">{evento.descricao}</p>}
        <ul className="flex flex-wrap gap-x-4 gap-y-1 text-sm text-muted-foreground">
          <li className="flex items-center gap-1.5"><Clock className="size-4" /> {hora}</li>
          <li className="flex items-center gap-1.5"><MapPin className="size-4" />
{evento.local}</li>
          <li className="flex items-center gap-1.5"><Users className="size-4" />
{evento.vagas} vagas</li>
        </ul>
      </div>

      <div className="col-span-2 flex items-center justify-between gap-3 sm:col-span-1
sm:flex-col sm:items-end">
        <Badge variant={ehGratuito(evento.valor) ? "secondary" : "default"}>
          {formatarValor(evento.valor)}
        </Badge>
      </div>
    </Card>
  )
}
```

O `Card` vem do shadcn: abra `src/components/ui/card.tsx` e veja que é uma função pequena com classes do Tailwind. Aqui ele recebe uma `className` que o transforma numa **grade de três colunas**: a data, o evento e o valor. No celular são duas colunas, e o valor desce para uma linha própria (`col-span-2`). Os ícones vêm do `lucide-react` , que o shadcn já instalou.

> **Conceito.** Props são **somente leitura**. O `EventoCard` usa o `evento` que recebeu, mas não o altera. Os dados descem do componente pai para o filho, nunca o contrário.

PASSO 3.5 · DADOS DE EXEMPLO

Antes de falar com a API, vamos desenhar a vitrine com três eventos fixos, copiados da resposta. Crie `src/dados-de-exemplo.ts` :

`src/dados-de-exemplo.ts`

```ts
import type { Evento } from "@/tipos"

// Três eventos copiados da resposta da API. Servem só até a Etapa 5,
// quando os dados passam a vir do servidor.
export const EVENTOS: Evento[] = [
  {
    id: 1,
    titulo: "Workshop de APIs com Laravel",
    descricao: "APIs REST, integração e webhooks na prática.",
    local: "Laboratório 3",
    data: "2026-12-05 19:00",
    vagas: 30,
    valor: "0.00",
    created_at: "2026-11-12T13:00:00.000000Z",
    updated_at: "2026-11-12T13:00:00.000000Z",
    user_id: null,
  },
  {
    id: 2,
    titulo: "Minicurso de Git e GitHub",
    descricao: "Versionamento de código do zero ao pull request.",
    local: "Auditório",
    data: "2026-12-12 14:00",
    vagas: 2,
    valor: "25.00",
    created_at: "2026-11-12T13:00:00.000000Z",
    updated_at: "2026-11-12T13:00:00.000000Z",
    user_id: null,
  },
  {
    id: 6,
    titulo: "Hackathon StartUB",
    descricao: null,
    local: "Bloco B",
    data: "2026-12-19 08:00",
    vagas: 60,
    valor: "40.00",
    created_at: "2026-11-12T13:00:00.000000Z",
    updated_at: "2026-11-12T13:00:00.000000Z",
    user_id: null,
  },
]
```

PASSO 3.6 · A VITRINE

Crie a pasta `src/pages` e, nela, `Vitrine.tsx`:

src/pages/Vitrine.tsx

```tsx
import { EventoCard } from "@/components/EventoCard"
import { EVENTOS } from "@/dados-de-exemplo"

export function Vitrine() {
  return (
    <section className="grid gap-8">
```

```
      <header className="grid gap-2">
        <h1 className="text-4xl font-extrabold sm:text-5xl">Programação do StartUB 2026</h1>
        <p className="max-w-prose text-lg text-muted-foreground">
          Oficinas, minicursos e o hackathon do congresso. Escolha um evento e faça sua
inscrição.
        </p>
      </header>

      <div className="grid gap-3">
        {EVENTOS.map((evento) => (
          <EventoCard key={evento.id} evento={evento} />
        ))}
      </div>
    </section>
  )
}
```

Por fim, troque **todo** o conteúdo de `src/App.tsx`:

src/App.tsx

```
import { Vitrine } from "@/pages/Vitrine"

export default function App() {
  return (
    <main className="mx-auto max-w-5xl px-4 py-8">
      <Vitrine />
    </main>
  )
}
```

Atenção. Todo item gerado por `map` precisa de uma `key` única e estável, como o `id`. Sem ela, o console mostra o aviso *Each child in a list should have a unique "key" prop*. Usar a posição na lista como chave parece funcionar, mas embaralha os cartões quando a lista é filtrada.

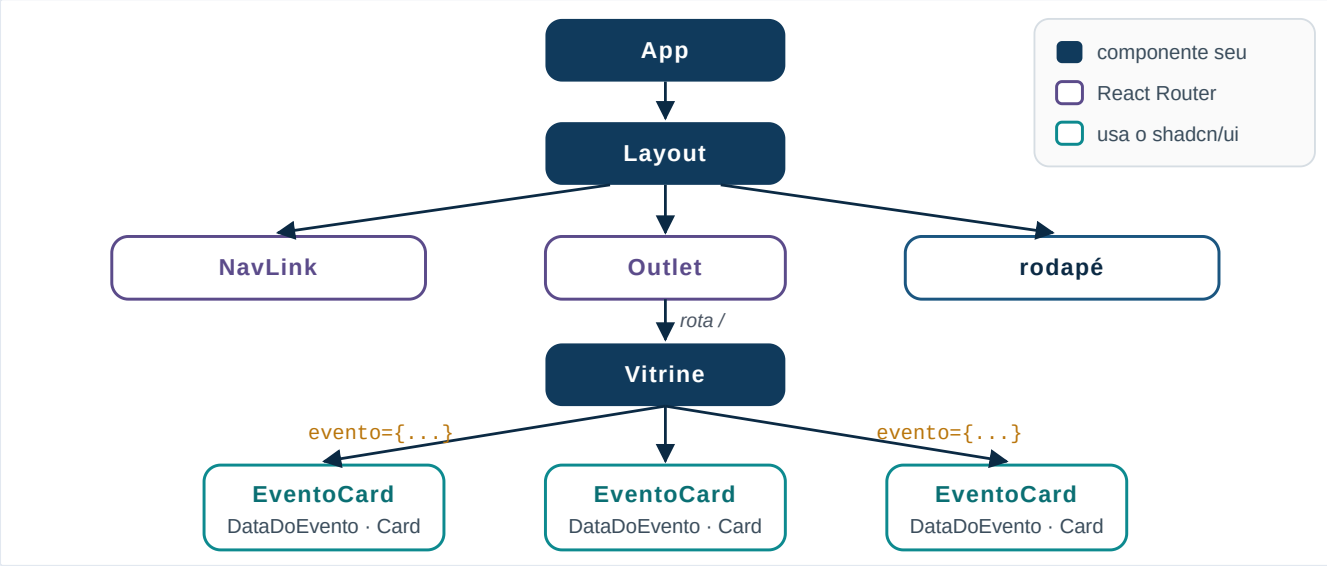


Figura 2. A árvore de componentes ao fim do Dia 1: os dados descem pelas props

> **CONFIRA ANTES DE SEGUIR.** A página mostra a programação com três eventos, cada um com o dia em número grande à esquerda, no gradiente violeta e magenta. As etiquetas são **Gratuito**, **R$ 25,00** e **R$ 40,00**, e os horários aparecem como *14h00*. Os títulos estão na fonte arredondada da logo. No DevTools, a aba **Components** mostra `App > Vitrine > EventoCard`, e clicar num `EventoCard` exibe a prop `evento`.

4. Rotas

Numa SPA, trocar de página não pede nada ao servidor. O **React Router** observa o endereço do navegador e escolhe qual componente desenhar. A Figura 3 mostra o esquema que vamos montar: um **layout** com cabeçalho e rodapé que nunca muda, e no meio dele, no `Outlet`, a página da rota atual.

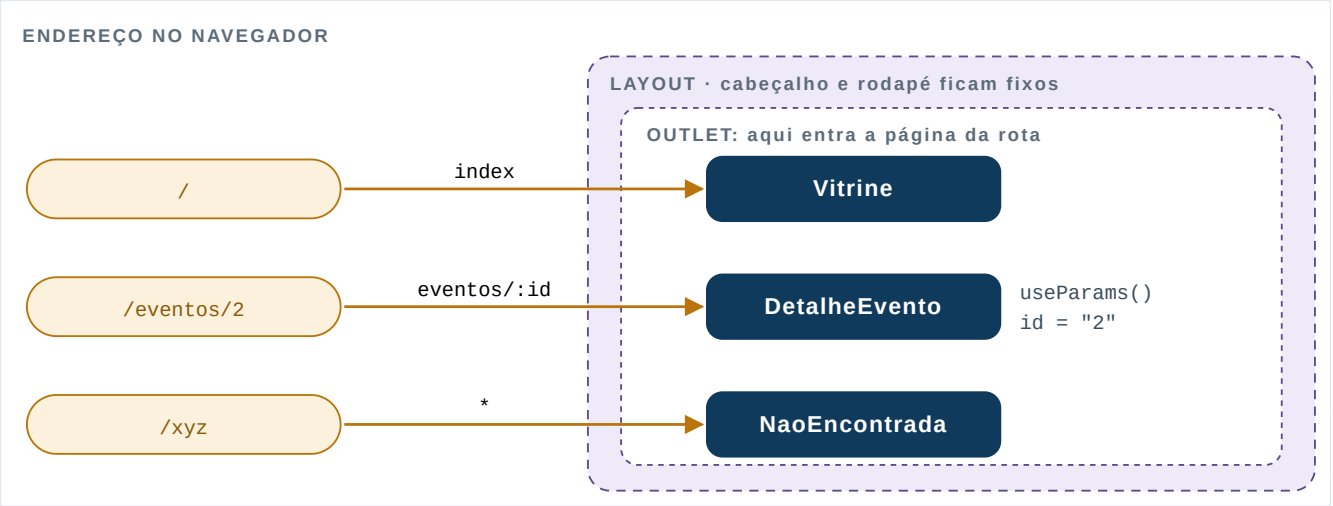


Figura 3. Cada endereço escolhe uma página, sempre dentro do mesmo layout

PASSO 4.1 · LIGAR O ROTEADOR

Em `src/main.tsx`, envolva o `App` com o `BrowserRouter`:

src/main.tsx

```
import { StrictMode } from "react"
import { createRoot } from "react-dom/client"
import { BrowserRouter } from "react-router"

import "./index.css"
import App from "./App.tsx"
import { ThemeProvider } from "@/components/theme-provider.tsx"

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <ThemeProvider>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </ThemeProvider>
```

```
  </StrictMode>
)
```

PASSO 4.2 · O LAYOUT

Crie `src/components/Layout.tsx`:

src/components/Layout.tsx

```tsx
import type { ReactNode } from "react"
import { NavLink, Outlet } from "react-router"
import { cn } from "@/lib/utils"

function Item({ para, children }: { para: string; children: ReactNode }) {
  return (
    <NavLink
      to={para}
      end
      className={({ isActive }) =>
        cn(
          "border-b-2 py-1 text-sm font-semibold",
          isActive ? "border-primary text-foreground" : "border-transparent text-muted-
foreground hover:text-foreground",
        )
      }
    >
      {children}
    </NavLink>
  )
}

export function Layout() {
  return (
    <div className="flex min-h-svh flex-col">
      <header className="sticky top-0 z-10 border-b bg-background/90 backdrop-blur">
        <div className="mx-auto flex max-w-5xl flex-wrap items-center gap-x-6 gap-y-2 px-4
py-3">
          <NavLink to="/" className="mr-auto">
            <img src="/startub-logo.jpg" alt="Congresso StartUB 2026" className="h-10 w-auto
rounded-md dark:bg-white dark:p-1" />
          </NavLink>
          <nav className="flex flex-wrap items-center gap-x-5 gap-y-1">
            <Item para="/">Programação</Item>
          </nav>
        </div>
      </header>

      <main className="mx-auto w-full max-w-5xl flex-1 px-4 py-10">
        <Outlet />
      </main>

      <footer className="onda-startub mt-10 pb-8 text-center text-sm">
        <p className="font-semibold">II Congresso de Ciência, Empreendedorismo e Inovação da
Unibalsas</p>
        <p className="opacity-75">Feito no minicurso de frontend com React e TypeScript.</p>
      </footer>
```

```
      </div>
    )
  }
```

O cabeçalho mostra a logo, que está em `public/`: tudo o que fica nessa pasta é servido pelo endereço da raiz, por isso o `src="/startub-logo.jpg"`. O `NavLink` é um link que sabe se está ativo: a função passada em `className` recebe `isActive` e devolve as classes, e o item da página atual ganha o sublinhado magenta. A função `cn()` junta as classes e resolve conflitos entre elas. O rodapé usa o utilitário `onda-startub`, que desenha as ondas das peças do congresso.

PASSO 4.3 · A PÁGINA 404

Crie `src/pages/NaoEncontrada.tsx`:

src/pages/NaoEncontrada.tsx

```tsx
import { Link } from "react-router"
import { Button } from "@/components/ui/button"

export function NaoEncontrada() {
  return (
    <section className="grid justify-items-center gap-4 py-16 text-center">
      <p className="texto-marca font-heading text-8xl font-extrabold">404</p>
      <h1 className="text-3xl font-bold">Esta página não existe</h1>
      <p className="text-muted-foreground">Confira o endereço ou volte para a programação.
</p>
      <Button asChild>
        <Link to="/">Ver a programação</Link>
      </Button>
    </section>
  )
}
```

PASSO 4.4 · A PÁGINA DO EVENTO

Crie `src/pages/DetalheEvento.tsx`:

src/pages/DetalheEvento.tsx

```tsx
import { Link, useParams } from "react-router"
import { ArrowLeft } from "lucide-react"
import { DataDoEvento } from "@/components/DataDoEvento"
import { Badge } from "@/components/ui/badge"
import { EVENTOS } from "@/dados-de-exemplo"
import { ehGratuito, formatarData, formatarValor } from "@/lib/formatar"
import { NaoEncontrada } from "@/pages/NaoEncontrada"

export function DetalheEvento() {
  const { id } = useParams()
  const evento = EVENTOS.find((e) => e.id === Number(id))

  if (!evento) return <NaoEncontrada />

  return (
```

```
    <article className="grid gap-6">
      <Link to="/" className="flex w-fit items-center gap-1 text-sm text-muted-foreground
hover:text-foreground">
        <ArrowLeft className="size-4" /> Voltar para a programação
      </Link>

      <div className="flex items-start gap-6">
        <DataDoEvento data={evento.data} className="pt-1" />
        <div className="grid gap-3">
          <Badge variant={ehGratuito(evento.valor) ? "secondary" : "default"}>
            {formatarValor(evento.valor)}
          </Badge>
          <h1 className="text-4xl leading-none font-extrabold sm:text-5xl">{evento.titulo}
</h1>
        </div>
      </div>

      {evento.descricao && <p className="max-w-prose text-lg text-muted-foreground">
{evento.descricao}</p>}

      <dl className="grid gap-4 rounded-xl bg-muted p-5 sm:grid-cols-3">
        <div>
          <dt className="text-sm text-muted-foreground">Quando</dt>
          <dd className="font-semibold">{formatarData(evento.data)}</dd>
        </div>
        <div>
          <dt className="text-sm text-muted-foreground">Onde</dt>
          <dd className="font-semibold">{evento.local}</dd>
        </div>
        <div>
          <dt className="text-sm text-muted-foreground">Vagas</dt>
          <dd className="font-semibold">{evento.vagas}</dd>
        </div>
      </dl>
    </article>
  )
}
```

A página reaproveita o `DataDoEvento` e organiza os dados numa **lista de definições** (`<dl>`, `<dt>`, `<dd>`), o elemento HTML feito para pares de nome e valor. O `useParams()` lê o pedaço variável do endereço. Na rota `eventos/:id`, o endereço `/eventos/2` entrega `{ id: "2" }`. É **texto**, por isso o `Number(id)` antes de comparar.

PASSO 4.5 · AS ROTAS

Troque o conteúdo de `src/App.tsx`:

`src/App.tsx`

```
import { Route, Routes } from "react-router"
import { Layout } from "@/components/Layout"
import { DetalheEvento } from "@/pages/DetalheEvento"
import { NaoEncontrada } from "@/pages/NaoEncontrada"
import { Vitrine } from "@/pages/Vitrine"
```

```tsx
export default function App() {
  return (
    <Routes>
      <Route element={<Layout />}>
        <Route index element={<Vitrine />} />
        <Route path="eventos/:id" element={<DetalheEvento />} />
        <Route path="*" element={<NaoEncontrada />} />
      </Route>
    </Routes>
  )
}
```

PASSO 4.6 · O BOTÃO "VER EVENTO"

Agora que existe roteador, o cartão pode ter um link. Atualize `src/components/EventoCard.tsx`:

src/components/EventoCard.tsx

```tsx
import { Link } from "react-router"
import { Clock, MapPin, Users } from "lucide-react"
import { DataDoEvento } from "@/components/DataDoEvento"
import { Badge } from "@/components/ui/badge"
import { Button } from "@/components/ui/button"
import { Card } from "@/components/ui/card"
import { ehGratuito, formatarValor, partesDaData } from "@/lib/formatar"
import type { Evento } from "@/tipos"

interface EventoCardProps {
  evento: Evento
}

// Uma linha da programação: a data à esquerda, o evento no meio, o valor e o botão à direita.
export function EventoCard({ evento }: EventoCardProps) {
  const { hora } = partesDaData(evento.data)

  return (
    <Card className="grid grid-cols-[4.5rem_1fr] gap-4 px-4 text-base sm:grid-cols-[5rem_1fr_auto] sm:items-center sm:px-6">
      <DataDoEvento data={evento.data} />

      <div className="grid gap-1.5">
        <h2 className="text-xl leading-tight font-bold">{evento.titulo}</h2>
        {evento.descricao && <p className="text-muted-foreground">{evento.descricao}</p>}
        <ul className="flex flex-wrap gap-x-4 gap-y-1 text-sm text-muted-foreground">
          <li className="flex items-center gap-1.5"><Clock className="size-4" /> {hora}</li>
          <li className="flex items-center gap-1.5"><MapPin className="size-4" /> {evento.local}</li>
          <li className="flex items-center gap-1.5"><Users className="size-4" /> {evento.vagas} vagas</li>
        </ul>
      </div>

      <div className="col-span-2 flex items-center justify-between gap-3 sm:col-span-1 sm:flex-col sm:items-end">
        <Badge variant={ehGratuito(evento.valor) ? "secondary" : "default"}>
```

```
          {formatarValor(evento.valor)}
        </Badge>
        <Button asChild>
          <Link to={`/eventos/${evento.id}`}>Ver evento</Link>
        </Button>
      </div>
    </Card>
  )
}
```

O `Button` do shadcn com `asChild` não desenha um `<button>`: ele passa o visual de botão para o filho, o `Link`. Fica com cara de botão e comportamento de link.

> **Atenção.** Use `Link`, e não `<a href>`, para navegar dentro da aplicação. O `<a>` faz o navegador baixar tudo de novo, e a página pisca. O `Link` só troca o componente. O `<a>` continua certo para endereços de fora, como o link de pagamento que veremos no segundo encontro.

> **CONFIRA ANTES DE SEGUIR.** O cabeçalho com a logo e o rodapé com as ondas aparecem em todas as páginas. **Ver evento** abre `/eventos/2`, com o título do evento, sem a página piscar. O botão voltar do navegador funciona. `/eventos/99` e `/qualquer-coisa` mostram o 404.

5. Cadastro e login

5.1 A API pede um crachá

Até aqui, a vitrine usa dados de exemplo. Antes de buscar os eventos de verdade, tente pedir a lista à API sem se identificar:

GET	https://startub.appmake.com.br/api/eventos	401 Unauthorized

RESPOSTA

```
{
  "message": "Unauthenticated."
}
```

A API do StartUB só atende quem tem conta. Os únicos endereços abertos são o de **cadastro** e o de **login**, e os dois devolvem a mesma coisa: os dados do usuário e um **token**.

POST	https://startub.appmake.com.br/api/register	201 Created

corpo enviado

```
{
  "name": "Ana Souza",
  "email": "ana.souza@example.com",
```

```
  "password": "segredo123",
  "password_confirmation": "segredo123"
}
```

RESPOSTA

```
{
  "user": {
    "name": "Ana Souza",
    "email": "ana.souza@example.com",
    "updated_at": "2026-11-12T20:21:21.000000Z",
    "created_at": "2026-11-12T20:21:21.000000Z",
    "id": 2
  },
  "token": "5|a4H1rW..."
}
```

Com a senha errada, o login responde com 422, e a mensagem já vem em português:

POST https://startub.appmake.com.br/api/login 422 Unprocessable Content

corpo enviado

```
{
  "email": "ana.souza@example.com",
  "password": "senha-errada"
}
```

RESPOSTA

```
{
  "message": "Credenciais inválidas.",
  "errors": {
    "email": [
      "Credenciais inválidas."
    ]
  }
}
```

Com a senha certa, devolve o usuário e um token novo:

POST https://startub.appmake.com.br/api/login 200 OK

corpo enviado

```
{
  "email": "ana.souza@example.com",
  "password": "segredo123"
}
```

RESPOSTA

```
{
  "user": {
    "id": 2,
    "name": "Ana Souza",
    "email": "ana.souza@example.com",
    "email_verified_at": null,
    "created_at": "2026-11-12T20:21:21.000000Z",
    "updated_at": "2026-11-12T20:21:21.000000Z"
  },
  "token": "6|9jImyy..."
}
```

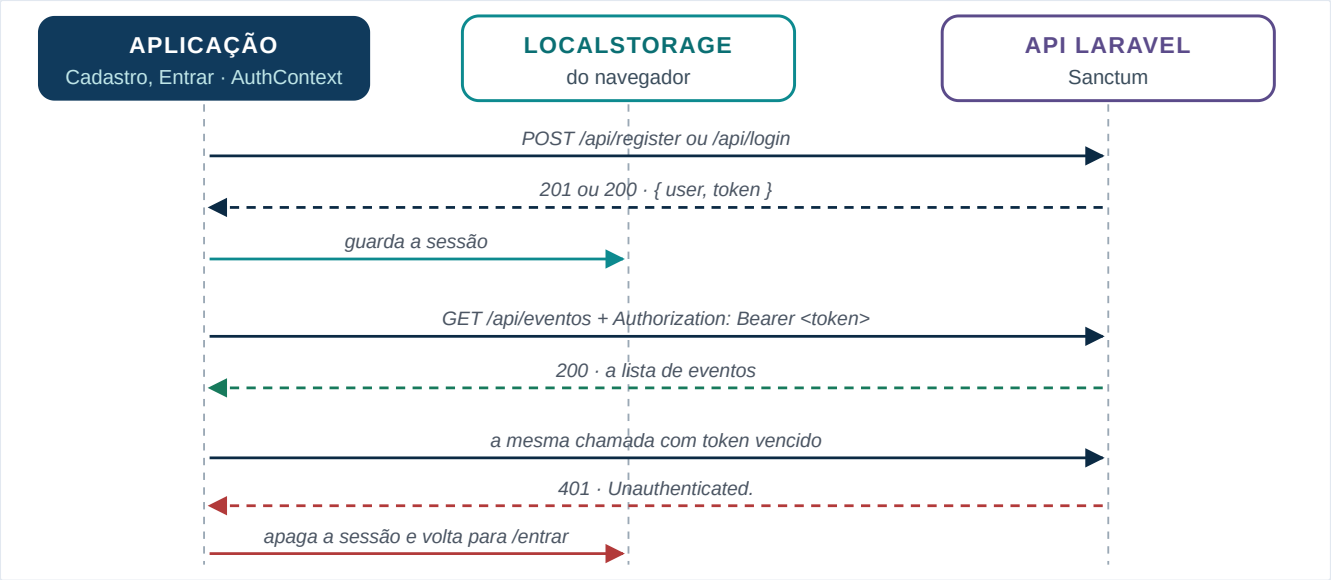


Figura 4. O token: recebido no cadastro ou no login, enviado em cada requisição, descartado no 401

> **Conceito.** O **token** funciona como a pulseira de um evento: quem a recebeu na entrada passa pelas portas sem mostrar documento de novo. A API não guarda sessão do navegador: cada requisição se identifica sozinha, levando o token no cabeçalho `Authorization: Bearer <token>`. Isso permite que o frontend rode num endereço e a API em outro, e que um aplicativo de celular use a mesma API do mesmo jeito. No Laravel, quem emite e confere esses tokens é o **Sanctum**.

O plano desta etapa: uma função que fala com a API e já manda o token; um **contexto** que guarda quem está logado; as páginas de cadastro e de login; e uma **rota protegida**, que só deixa ver a vitrine quem entrou.

PASSO 5.1 · O TIPO DO USUÁRIO

Atualize `src/tipos.ts`, acrescentando a interface `Usuario`:

src/tipos.ts

```
// O formato dos dados, copiado das respostas reais da API.

export interface Evento {
  id: number
  titulo: string
```

```
  descricao: string | null
  local: string
  data: string // "2026-12-05 19:00", sem o T
  vagas: number
  valor: string // "25.00", texto e não número
  created_at: string
  updated_at: string
  user_id: number | null // quem criou; null nos eventos de exemplo da API
}

export interface Usuario {
  id: number
  name: string
  email: string
}
```

PASSO 5.2 · A FUNÇÃO QUE FALA COM A API

Crie a pasta `src/services` e, nela, `api.ts`:

src/services/api.ts

```
export const API_URL =
  import.meta.env.VITE_API_URL ?? "https://startub.appmake.com.br/api"

export const CHAVE_SESSAO = "startub:sessao"

// Erro com o que a API respondeu: o status HTTP, a mensagem e,
// no caso do 422, os erros de cada campo.
export class ApiError extends Error {
  status: number
  erros: Record<string, string[]>

  constructor(status: number, mensagem: string, erros: Record<string, string[]> = {}) {
    super(mensagem)
    this.status = status
    this.erros = erros
  }
}

function tokenSalvo(): string | null {
  const sessao = localStorage.getItem(CHAVE_SESSAO)
  return sessao ? JSON.parse(sessao).token : null
}

export async function requisicao<T>(caminho: string, opcoes: RequestInit = {}): Promise<T> {
  const token = tokenSalvo()

  let resposta: Response
  try {
    resposta = await fetch(`${API_URL}${caminho}`, {
      ...opcoes,
      headers: {
        Accept: "application/json",
        "Content-Type": "application/json",
        ...(token ? { Authorization: `Bearer ${token}` } : {}),
```

```
        ...opcoes.headers,
      },
    })
  } catch {
    throw new ApiError(0, "Não foi possível falar com a API. Confira sua conexão.")
  }

  if (resposta.status === 204) return undefined as T

  const corpo = await resposta.json().catch(() => ({}))

  if (!resposta.ok) {
    if (resposta.status === 401 && token) {
      localStorage.removeItem(CHAVE_SESSAO)
      window.dispatchEvent(new Event("sessao-expirada"))
    }
    throw new ApiError(resposta.status, corpo.message ?? `Erro ${resposta.status}`,
corpo.errors)
  }

  return corpo as T
}
```

Esta é a peça mais importante do projeto. Toda conversa com a API passa por ela:

- o endereço vem de `VITE_API_URL`, se existir, e senão usa a API publicada. Para usar a API de ensaio, crie na raiz do projeto um arquivo `.env.local` com a linha `VITE_API_URL=http://localhost:8000/api` e reinicie o `npm run dev`;
- se há uma sessão guardada, o token vai no cabeçalho `Authorization` de **toda** requisição;
- o `fetch` **não** considera erro uma resposta 404 ou 422, só a falta de rede. Por isso o código confere `resposta.ok` e lança um `ApiError` com o status, a mensagem e os erros de cada campo;
- uma resposta **401** com token quer dizer que ele deixou de valer: a sessão é apagada, e o evento `sessao-expirada` avisa o resto da aplicação;
- o `<T>` em `requisicao<T>` é um **genérico**: quem chama diz o tipo que espera receber.

> **Atenção.** O template deste projeto proíbe declarar campos direto no construtor (`constructor(public status: number)`), por causa da opção `erasableSyntaxOnly` do `tsconfig`. Por isso o `ApiError` declara `status` e `erros` em cima e atribui no construtor. Se copiar código da internet com o atalho, o editor acusa *This syntax is not allowed when 'erasableSyntaxOnly' is enabled.*

PASSO 5.3 · O SERVIÇO DE AUTENTICAÇÃO

Crie `src/services/auth.ts`:

`src/services/auth.ts`

```
import { requisicao } from "./api"
import type { Usuario } from "@/tipos"

// O que a API devolve no cadastro e no login: o usuário e o token.
```

```ts
export interface Sessao {
  user: Usuario
  token: string
}

export interface DadosCadastro {
  name: string
  email: string
  password: string
  password_confirmation: string
}

export interface DadosLogin {
  email: string
  password: string
}

export function cadastrar(dados: DadosCadastro) {
  return requisicao<Sessao>("/register", {
    method: "POST",
    body: JSON.stringify(dados),
  })
}

export function entrar(dados: DadosLogin) {
  return requisicao<Sessao>("/login", {
    method: "POST",
    body: JSON.stringify(dados),
  })
}

export function sair() {
  return requisicao<void>("/logout", { method: "POST" })
}
```

As interfaces `DadosCadastro` e `DadosLogin` usam os nomes de campo **da API**, em inglês, como `password_confirmation`. Traduzir os nomes aqui obrigaria a converter de volta em cada envio.

5.2 Quem está logado: o primeiro Context

Várias partes da aplicação precisam saber quem está logado: o cabeçalho, para mostrar o botão **Sair**; a rota protegida, para decidir se deixa passar; as páginas de login e cadastro. Passar o usuário por props de componente em componente seria um caminho longo.

> **Conceito.** O **Context** do React disponibiliza um valor para todos os componentes abaixo de um *Provider*, sem passar por props em cada nível. Ele serve para o pouco estado que é da aplicação inteira: usuário logado, tema, preferências. Quem precisa do valor chama um hook, aqui o `useAuth()`.

PASSO 5.4 · O CONTEXTO DE AUTENTICAÇÃO

Crie a pasta `src/contexts` e, nela, `AuthContext.tsx`:

src/contexts/AuthContext.tsx

```tsx
import { createContext, useContext, useEffect, useState, type ReactNode } from "react"
import { CHAVE_SESSAO } from "@/services/api"
import * as auth from "@/services/auth"
import type { DadosCadastro, DadosLogin, Sessao } from "@/services/auth"
import type { Usuario } from "@/tipos"

interface AuthContextValor {
  usuario: Usuario | null
  cadastrar: (dados: DadosCadastro) => Promise<void>
  entrar: (dados: DadosLogin) => Promise<void>
  sair: () => Promise<void>
}

const AuthContext = createContext<AuthContextValor | null>(null)

function sessaoSalva(): Sessao | null {
  try {
    const texto = localStorage.getItem(CHAVE_SESSAO)
    return texto ? JSON.parse(texto) : null
  } catch {
    return null
  }
}

export function AuthProvider({ children }: { children: ReactNode }) {
  const [usuario, setUsuario] = useState<Usuario | null>(() => sessaoSalva()?.user ?? null)

  // A camada da API avisa quando o token deixou de valer (resposta 401).
  useEffect(() => {
    const aoExpirar = () => setUsuario(null)
    window.addEventListener("sessao-expirada", aoExpirar)
    return () => window.removeEventListener("sessao-expirada", aoExpirar)
  }, [])

  // Cadastro e login devolvem a mesma coisa: guardamos a sessão do mesmo jeito.
  function guardar(sessao: Sessao) {
    localStorage.setItem(CHAVE_SESSAO, JSON.stringify(sessao))
    setUsuario(sessao.user)
  }

  async function cadastrar(dados: DadosCadastro) {
    guardar(await auth.cadastrar(dados))
  }

  async function entrar(dados: DadosLogin) {
    guardar(await auth.entrar(dados))
  }

  async function sair() {
    try {
      await auth.sair()
    } finally {
      localStorage.removeItem(CHAVE_SESSAO)
      setUsuario(null)
```

```
    }
  }

  return (
    <AuthContext.Provider value={{ usuario, cadastrar, entrar, sair }}>
      {children}
    </AuthContext.Provider>
  )
}

export function useAuth() {
  const valor = useContext(AuthContext)
  if (!valor) throw new Error("useAuth precisa estar dentro de <AuthProvider>")
  return valor
}
```

Cadastro e login devolvem a mesma `Sessao`, então os dois terminam em `guardar`: a sessão vai para o `localStorage` e o usuário para o estado. O `useState` começa com uma **função**, `() => sessaoSalva()?.user ?? null`: quem recarrega a página continua logado, porque a sessão é lida do `localStorage` na abertura.

> **Atenção.** Guardar o token no `localStorage` é simples e comum, mas qualquer script que rode na página consegue lê-lo. Por isso o frontend nunca deve exibir HTML vindo de fora sem tratamento, e o backend deve dar ao token uma validade. É uma troca consciente entre simplicidade e segurança.

5.3 Formulários com useState

As páginas de cadastro e login são **componentes controlados**: o valor de cada campo mora num `useState`, o `input` mostra esse valor e, a cada tecla, o `onChange` atualiza o estado. O React passa a ser a única fonte da verdade do formulário.

PASSO 5.5 · TRADUZIR AS MENSAGENS DA API

A API valida o cadastro e responde em inglês, como *The password field must be at least 8 characters.* Crie `src/lib/erros.ts`:

`src/lib/erros.ts`

```
// A API responde as validações em inglês. Esta tabela traduz as mensagens
// que os formulários podem receber. O que não estiver aqui aparece como veio.
const TRADUCOES: [RegExp, string][] = [
  [/has already been taken/, "Este e-mail já está em uso."],
  [/field is required/, "Campo obrigatório."],
  [/must be a valid email/, "Informe um e-mail válido."],
  [/must be at least (\d+) characters/, "Use pelo menos $1 caracteres."],
  [/must not be greater than (\d+) characters/, "Use no máximo $1 caracteres."],
  [/confirmation does not match/, "As senhas não conferem."],
  [/Too Many Attempts/, "Muitas tentativas. Espere um minuto e tente de novo."],
  [/This action is unauthorized/, "Você não tem permissão para isso: só quem criou pode
alterar."],
  [/No query results/, "Não encontramos o que você procurou."],
]
```

```
export function traduzir(mensagem: string): string {
  for (const [padrao, traducao] of TRADUCOES) {
    const achou = mensagem.match(padrao)
    if (achou) return traducao.replace("$1", achou[1] ?? "")
  }
  return mensagem
}
```

PASSO 5.6 · A MOLDURA DAS TELAS DE ACESSO

Cadastro e login são as primeiras telas que alguém vê, e as duas têm o mesmo desenho: as boas-vindas à esquerda, com o "2026" grande no gradiente das peças do congresso, e o formulário à direita. Esse desenho comum vira um componente que **recebe outro componente** como conteúdo, pela prop especial `children`. Crie `src/components/TelaDeAcesso.tsx`:

`src/components/TelaDeAcesso.tsx`

```
import type { ReactNode } from "react"

// Moldura das telas de cadastro e de login: as boas-vindas à esquerda,
// o formulário à direita. No celular, só o formulário.
export function TelaDeAcesso({ children }: { children: ReactNode }) {
  return (
    <div className="grid items-center gap-10 lg:grid-cols-[1fr_24rem]">
      <div className="hidden gap-4 lg:grid">
        <p className="texto-marca font-heading text-8xl leading-none font-
extrabold">2026</p>
        <h1 className="max-w-lg text-4xl leading-tight font-extrabold">
          As inscrições do congresso StartUB, num lugar só
        </h1>
        <p className="max-w-prose text-lg text-muted-foreground">
          Veja a programação, inscreva-se nas oficinas, nos minicursos e no hackathon e
acompanhe o
          pagamento com a sua conta.
        </p>
      </div>
      {children}
    </div>
  )
}
```

O tipo `ReactNode` aceita qualquer coisa que o React sabe desenhar. Quem usa escreve `<TelaDeAcesso>...o formulário...</TelaDeAcesso>`, e o que estiver entre as tags chega como `children`. No celular, a coluna das boas-vindas some (`hidden lg:grid`) e fica só o formulário.

PASSO 5.7 · A PÁGINA DE CADASTRO

Crie `src/pages/Cadastro.tsx`:

`src/pages/Cadastro.tsx`

```tsx
import { useState, type FormEvent } from "react"
import { Link, Navigate } from "react-router"
import { TelaDeAcesso } from "@/components/TelaDeAcesso"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Field, FieldError, FieldGroup, FieldLabel } from "@/components/ui/field"
import { Input } from "@/components/ui/input"
import { useAuth } from "@/contexts/AuthContext"
import { traduzir } from "@/lib/erros"
import { ApiError } from "@/services/api"

export function Cadastro() {
  const { usuario, cadastrar } = useAuth()

  // Um único useState com um objeto para os quatro campos
  const [dados, setDados] = useState({ name: "", email: "", password: "",
password_confirmation: "" })
  const [erros, setErros] = useState<Record<string, string[]>>({})
  const [enviando, setEnviando] = useState(false)

  if (usuario) return <Navigate to="/" replace />

  // Troca só o campo que mudou, mantendo os outros
  function mudar(campo: keyof typeof dados, valor: string) {
    setDados((atual) => ({ ...atual, [campo]: valor }))
  }

  async function enviar(e: FormEvent<HTMLFormElement>) {
    e.preventDefault()
    setEnviando(true)
    setErros({})
    try {
      await cadastrar(dados)
    } catch (erro) {
      if (erro instanceof ApiError && erro.status === 422) setErros(erro.erros)
      else setErros({ geral: [(erro as Error).message] })
    } finally {
      setEnviando(false)
    }
  }

  function erroDe(campo: string) {
    const mensagem = erros[campo]?.[0]
    return mensagem ? [{ message: traduzir(mensagem) }] : undefined
  }

  return (
    <TelaDeAcesso>
      <Card className="w-full max-w-sm justify-self-center">
        <CardHeader>
          <CardTitle className="text-3xl font-bold">Criar conta</CardTitle>
          <CardDescription>Crie sua conta para ver a programação do StartUB 2026 e se
inscrever.</CardDescription>
        </CardHeader>
        <CardContent>
```

```
          <form onSubmit={enviar}>
            <FieldGroup>
              <Field data-invalid={!!erros.name}>
                <FieldLabel htmlFor="name">Nome</FieldLabel>
                <Input id="name" value={dados.name} onChange={(e) => mudar("name",
e.target.value)} />
                <FieldError errors={erroDe("name")} />
              </Field>
              <Field data-invalid={!!erros.email}>
                <FieldLabel htmlFor="email">E-mail</FieldLabel>
                <Input id="email" type="email" value={dados.email} onChange={(e) =>
mudar("email", e.target.value)} />
                <FieldError errors={erroDe("email")} />
              </Field>
              <Field data-invalid={!!erros.password}>
                <FieldLabel htmlFor="password">Senha</FieldLabel>
                <Input id="password" type="password" value={dados.password} onChange={(e) =>
mudar("password", e.target.value)} />
                <FieldError errors={erroDe("password")} />
              </Field>
              <Field>
                <FieldLabel htmlFor="password_confirmation">Confirme a senha</FieldLabel>
                <Input id="password_confirmation" type="password" value=
{dados.password_confirmation} onChange={(e) => mudar("password_confirmation",
e.target.value)} />
              </Field>
              <FieldError errors={erroDe("geral")} />
              <Button type="submit" disabled={enviando}>
                {enviando ? "Criando..." : "Criar conta"}
              </Button>
              <p className="text-center text-sm text-muted-foreground">
                Já tem conta? <Link to="/entrar" className="text-primary
underline">Entrar</Link>
              </p>
            </FieldGroup>
          </form>
        </CardContent>
      </Card>
    </TelaDeAcesso>
  )
}
```

Os pontos novos:

- um único `useState` guarda os quatro campos num objeto, e a função `mudar` troca só o campo que mudou, copiando os outros com `...atual`;
- `e.preventDefault()` impede o navegador de recarregar a página ao enviar o formulário, que é o comportamento padrão do HTML;
- `try`, `catch` e `finally`: se a API recusar, o `catch` guarda os erros; com ou sem erro, o `finally` libera o botão;
- depois do cadastro, o contexto passa a ter um usuário, e a linha `if (usuario) return <Navigate ... />` leva para a vitrine. Ninguém precisa mandar navegar.

PASSO 5.8 · A PÁGINA DE LOGIN

Crie `src/pages/Entrar.tsx`:

src/pages/Entrar.tsx

```tsx
import { useState, type FormEvent } from "react"
import { Link, Navigate, useLocation } from "react-router"
import { TelaDeAcesso } from "@/components/TelaDeAcesso"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card"
import { Field, FieldError, FieldGroup, FieldLabel } from "@/components/ui/field"
import { Input } from "@/components/ui/input"
import { useAuth } from "@/contexts/AuthContext"
import { traduzir } from "@/lib/erros"
import { ApiError } from "@/services/api"

export function Entrar() {
  const { usuario, entrar } = useAuth()
  const local = useLocation()
  const destino = local.state?.voltarPara ?? "/"

  // Componentes controlados: o valor de cada campo mora num useState.
  const [email, setEmail] = useState("")
  const [password, setPassword] = useState("")
  const [erros, setErros] = useState<Record<string, string[]>>({})
  const [enviando, setEnviando] = useState(false)

  // Depois do login, o usuário existe e esta linha leva para o destino.
  if (usuario) return <Navigate to={destino} replace />

  async function enviar(e: FormEvent<HTMLFormElement>) {
    e.preventDefault() // o navegador não recarrega a página
    setEnviando(true)
    setErros({})
    try {
      await entrar({ email, password })
    } catch (erro) {
      // 422 traz um erro por campo; qualquer outro vira um erro geral
      if (erro instanceof ApiError && erro.status === 422) setErros(erro.erros)
      else setErros({ geral: [(erro as Error).message] })
    } finally {
      setEnviando(false)
    }
  }

  // O primeiro erro do campo, traduzido, no formato que o FieldError espera
  function erroDe(campo: string) {
    const mensagem = erros[campo]?.[0]
    return mensagem ? [{ message: traduzir(mensagem) }] : undefined
  }

  return (
    <TelaDeAcesso>
      <Card className="w-full max-w-sm justify-self-center">
        <CardHeader>
```

```
          <CardTitle className="text-3xl font-bold">Entrar</CardTitle>
          <CardDescription>Entre para ver a programação e fazer suas inscrições.
</CardDescription>
        </CardHeader>
        <CardContent>
          <form onSubmit={enviar}>
            <FieldGroup>
              <Field data-invalid={!!erros.email}>
                <FieldLabel htmlFor="email">E-mail</FieldLabel>
                <Input id="email" type="email" value={email} onChange={(e) =>
setEmail(e.target.value)} />
                <FieldError errors={erroDe("email")} />
              </Field>
              <Field data-invalid={!!erros.password}>
                <FieldLabel htmlFor="password">Senha</FieldLabel>
                <Input id="password" type="password" value={password} onChange={(e) =>
setPassword(e.target.value)} />
                <FieldError errors={erroDe("password")} />
              </Field>
              <FieldError errors={erroDe("geral")} />
              <Button type="submit" disabled={enviando}>
                {enviando ? "Entrando..." : "Entrar"}
              </Button>
              <p className="text-center text-sm text-muted-foreground">
                Ainda não tem conta? <Link to="/cadastro" className="text-primary
underline">Criar conta</Link>
              </p>
            </FieldGroup>
          </form>
        </CardContent>
      </Card>
    </TelaDeAcesso>
  )
}
```

O `useLocation().state?.voltarPara` é o endereço que a pessoa tentou abrir antes de ser mandada para o login. Depois de entrar, ela volta para lá.

PASSO 5.9 · A ROTA PROTEGIDA

Crie `src/components/RotaProtegida.tsx`:

src/components/RotaProtegida.tsx

```tsx
import { Navigate, Outlet, useLocation } from "react-router"
import { useAuth } from "@/contexts/AuthContext"

// Sem usuário logado, manda para /entrar e lembra de onde a pessoa veio.
export function RotaProtegida() {
  const { usuario } = useAuth()
  const local = useLocation()

  if (!usuario) {
    return <Navigate to="/entrar" replace state={{ voltarPara: local.pathname }} />
  }
```

```
  return <Outlet />
}
```

A rota protegida não desenha nada próprio: se há usuário, deixa passar para o `Outlet`; se não há, redireciona para `/entrar` e anota no `state` de onde a pessoa veio.

PASSO 5.10 · ROTAS, MENU E PROVIDER

Atualize `src/App.tsx`. Cadastro e login ficam abertos; todo o resto vai para dentro da rota protegida:

src/App.tsx

```tsx
import { Route, Routes } from "react-router"
import { Layout } from "@/components/Layout"
import { RotaProtegida } from "@/components/RotaProtegida"
import { Cadastro } from "@/pages/Cadastro"
import { DetalheEvento } from "@/pages/DetalheEvento"
import { Entrar } from "@/pages/Entrar"
import { NaoEncontrada } from "@/pages/NaoEncontrada"
import { Vitrine } from "@/pages/Vitrine"

export default function App() {
  return (
    <Routes>
      <Route element={<Layout />}>
        {/* abertas: quem ainda não entrou */}
        <Route path="entrar" element={<Entrar />} />
        <Route path="cadastro" element={<Cadastro />} />

        {/* todo o resto exige login, porque a API exige token */}
        <Route element={<RotaProtegida />}>
          <Route index element={<Vitrine />} />
          <Route path="eventos/:id" element={<DetalheEvento />} />
        </Route>

        <Route path="*" element={<NaoEncontrada />} />
      </Route>
    </Routes>
  )
}
```

Atualize `src/components/Layout.tsx`, com **Entrar** e **Criar conta** para quem não está logado e **Sair** para quem está:

src/components/Layout.tsx

```tsx
import type { ReactNode } from "react"
import { NavLink, Outlet } from "react-router"
import { LogOut } from "lucide-react"
import { Button } from "@/components/ui/button"
import { useAuth } from "@/contexts/AuthContext"
import { cn } from "@/lib/utils"

function Item({ para, children }: { para: string; children: ReactNode }) {
```

```
  return (
    <NavLink
      to={para}
      end
      className={({ isActive }) =>
        cn(
          "border-b-2 py-1 text-sm font-semibold",
          isActive ? "border-primary text-foreground" : "border-transparent text-muted-
foreground hover:text-foreground",
        )
      }
    >
      {children}
    </NavLink>
  )
}

export function Layout() {
  const { usuario, sair } = useAuth()

  return (
    <div className="flex min-h-svh flex-col">
      <header className="sticky top-0 z-10 border-b bg-background/90 backdrop-blur">
        <div className="mx-auto flex max-w-5xl flex-wrap items-center gap-x-6 gap-y-2 px-4
py-3">
          <NavLink to="/" className="mr-auto">
            <img src="/startub-logo.jpg" alt="Congresso StartUB 2026" className="h-10 w-auto
rounded-md dark:bg-white dark:p-1" />
          </NavLink>
          <nav className="flex flex-wrap items-center gap-x-5 gap-y-1">
            {usuario ? (
              <>
                <Item para="/">Programação</Item>
                <Button variant="ghost" size="sm" onClick={sair} title={`Sair da conta de
${usuario.name}`}>
                  <LogOut /> <span className="sr-only sm:not-sr-only">Sair</span>
                </Button>
              </>
            ) : (
              <>
                <Item para="/entrar">Entrar</Item>
                <Item para="/cadastro">Criar conta</Item>
              </>
            )}
          </nav>
        </div>
      </header>

      <main className="mx-auto w-full max-w-5xl flex-1 px-4 py-10">
        <Outlet />
      </main>

      <footer className="onda-startub mt-10 pb-8 text-center text-sm">
        <p className="font-semibold">II Congresso de Ciência, Empreendedorismo e Inovação da
Unibalsas</p>
        <p className="opacity-75">Feito no minicurso de frontend com React e TypeScript.</p>
```

```
      </footer>
    </div>
  )
}
```

E envolva o `App` com o `AuthProvider` em `src/main.tsx`:

src/main.tsx

```tsx
import { StrictMode } from "react"
import { createRoot } from "react-dom/client"
import { BrowserRouter } from "react-router"

import "./index.css"
import App from "./App.tsx"
import { ThemeProvider } from "@/components/theme-provider.tsx"
import { AuthProvider } from "@/contexts/AuthContext"

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <ThemeProvider>
      <BrowserRouter>
        <AuthProvider>
          <App />
        </AuthProvider>
      </BrowserRouter>
    </ThemeProvider>
  </StrictMode>
)
```



Figura 5. O cadastro recusado pela API: cada erro no seu campo, já traduzido

Atenção. O login aceita **6 tentativas por minuto** por endereço de internet, e uma turma inteira na mesma rede sai para a internet pelo mesmo endereço. Se muita gente errar a senha ao mesmo tempo, todos recebem:

`429 Too Many Requests`

com a mensagem *Too Many Attempts.*, que a nossa tradução mostra como **Muitas tentativas. Espere um minuto e tente de novo.** Por isso, na aula, crie a conta pelo **cadastro**, que não tem esse limite e já entra logado, e só use o login quando precisar.

CONFIRA ANTES DE SEGUIR. Abrir `http://localhost:5173` sem estar logado leva para `/entrar`. O cadastro com senhas diferentes mostra **As senhas não conferem.**; com um e-mail já usado, **Este e-mail já está em uso.** Um cadastro válido leva direto para a vitrine, com **Sair** no menu. Recarregar a página mantém o login. No DevTools, em **Application → Local Storage**, aparece a chave `startub:sessao`. **Sair** apaga essa chave e volta para o login.

6. Falando com a API

6.1 Olhando a resposta crua

Com o token, a API responde. A aba **Network** (Rede) do DevTools mostra cada requisição que a aplicação fizer daqui em diante. Esta é a lista de eventos:

GET `https://startub.appmake.com.br/api/eventos` **200 OK**

RESPOSTA


```
[
  {
    "id": 1,
    "titulo": "Workshop de APIs com Laravel",
    "descricao": "APIs REST, integração e webhooks na prática.",
    "local": "Laboratório 3",
    "data": "2026-12-05 19:00",
    "vagas": 30,
    "valor": "0.00",
    "created_at": "2026-11-12T16:01:24.000000Z",
    "updated_at": "2026-11-12T16:01:24.000000Z",
    "user_id": null
  },
  ... mais 3 eventos
]
```


E o caso de erro, um evento que não existe:

GET https://startub.appmake.com.br/api/eventos/999 404 Not Found

RESPOSTA

```
{
  "message": "No query results for model [App\\Models\\Evento] 999"
}
```

A API responde com um **código de status** e um corpo em JSON. Os que vamos encontrar estão na Tabela 2.

Status	Significado	Quando aparece
200 OK	deu certo	consultas e login
201 Created	criado	cadastro, inscrição ou evento novo
401 Unauthorized	sem token, ou token inválido	qualquer rota fora o cadastro e o login
403 Forbidden	o token vale, mas a conta não pode mexer ali	alterar o evento ou ver a inscrição de outra pessoa
404 Not Found	não existe	id errado
409 Conflict	conflito com o estado atual	evento lotado
422 Unprocessable Content	dados inválidos	formulário com erro, senha errada
429 Too Many Requests	tentativas demais	login repetido muitas vezes

> **Na prática.** A aba Network é o melhor amigo de quem faz frontend. Quando algo não aparece na tela, a primeira pergunta é: a requisição saiu? Voltou com que status? Levou o token? Metade dos problemas se resolve olhando ali, antes de olhar o código.

6.2 O dado do servidor é diferente

A forma mais óbvia de buscar dados em React é um `useEffect` com `fetch` que guarda o resultado num `useState`. Funciona no primeiro exemplo e se complica depressa: é preciso guardar à parte o *carregando* e o *erro*, evitar buscar de novo o que já foi buscado, atualizar a lista quando algo muda em outra tela, cancelar a resposta que chega depois que o usuário já saiu da página.

> **Conceito.** Dado que vem do servidor não pertence à tela: ele é uma **cópia** de algo que mora em outro lugar e pode mudar sem aviso. O **TanStack Query** é a biblioteca que cuida dessa cópia: busca, guarda num **cache** identificado por uma **chave**, informa se está carregando ou se deu erro, e sabe quando buscar de novo.

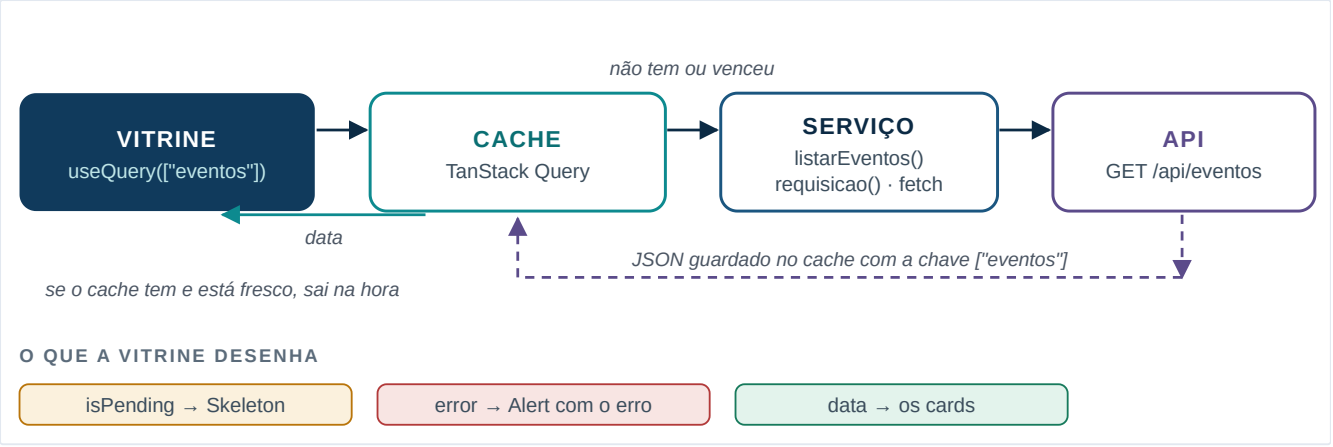


Figura 6. O caminho de uma consulta: a chave identifica o dado no cache

PASSO 6.1 · O SERVIÇO DE EVENTOS

Crie `src/services/eventos.ts`:

src/services/eventos.ts

```
import type { Evento } from "@/tipos"
import { requisicao } from "./api"

export function listarEventos() {
  return requisicao<Evento[]>("/eventos")
}

export function buscarEvento(id: number) {
  return requisicao<Evento>(`/eventos/${id}`)
}
```

Repare que nada aqui fala de token: a função `requisicao` do Passo 5.2 já cuida disso.

PASSO 6.2 · LIGAR O TANSTACK QUERY

Atualize `src/main.tsx`:

src/main.tsx

```
import { StrictMode } from "react"
import { createRoot } from "react-dom/client"
import { BrowserRouter } from "react-router"
import { QueryClient, QueryClientProvider } from "@tanstack/react-query"
import { ReactQueryDevtools } from "@tanstack/react-query-devtools"

import "./index.css"
import App from "./App.tsx"
import { ThemeProvider } from "@/components/theme-provider.tsx"
import { AuthProvider } from "@/contexts/AuthContext"

const queryClient = new QueryClient({
  defaultOptions: {
    queries: { staleTime: 30_000 }, // dados valem 30 s antes de buscar de novo
  },
```

```
})

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <ThemeProvider>
      <QueryClientProvider client={queryClient}>
        <BrowserRouter>
          <AuthProvider>
            <App />
          </AuthProvider>
        </BrowserRouter>
        <ReactQueryDevtools />
      </QueryClientProvider>
    </ThemeProvider>
  </StrictMode>
)
```

O `QueryClient` é o cache. O `staleTime` diz por quanto tempo um dado é considerado fresco: dentro dos 30 segundos, voltar à vitrine não faz nova requisição. O `ReactQueryDevtools` põe um botão redondo no canto da tela, que só aparece em desenvolvimento.

PASSO 6.3 · OS TRÊS ESTADOS DE UMA TELA

Toda tela que busca dados precisa desenhar três situações além do sucesso. Crie `src/components/Estados.tsx`:

src/components/Estados.tsx

```tsx
import type { ReactNode } from "react"
import { AlertCircle } from "lucide-react"
import { Alert, AlertDescription, AlertTitle } from "@/components/ui/alert"
import { Button } from "@/components/ui/button"
import { Skeleton } from "@/components/ui/skeleton"
import { traduzir } from "@/lib/erros"

// Os três estados que toda tela que busca dados precisa desenhar:
// carregando, erro e vazio.

export function CarregandoCards() {
  return (
    <div className="grid gap-3">
      {[1, 2, 3].map((n) => (
        <Skeleton key={n} className="h-32 rounded-xl" />
      ))}
    </div>
  )
}

interface MensagemErroProps {
  erro: Error
  tentarDeNovo?: () => void
}

export function MensagemErro({ erro, tentarDeNovo }: MensagemErroProps) {
  return (
    <Alert variant="destructive">
```

```
      <AlertCircle />
      <AlertTitle>Algo deu errado</AlertTitle>
      <AlertDescription>
        <p>{traduzir(erro.message)}</p>
        {tentarDeNovo && (
          <Button variant="outline" size="sm" className="mt-2" onClick={tentarDeNovo}>
            Tentar de novo
          </Button>
        )}
      </AlertDescription>
    </Alert>
  )
}

export function Vazio({ children }: { children: ReactNode }) {
  return (
    <p className="rounded-xl border-2 border-dashed p-10 text-center text-muted-foreground">
      {children}
    </p>
  )
}
```

PASSO 6.4 · A VITRINE COM DADOS REAIS

Atualize `src/pages/Vitrine.tsx`:

src/pages/Vitrine.tsx

```
import { useQuery } from "@tanstack/react-query"
import { EventoCard } from "@/components/EventoCard"
import { CarregandoCards, MensagemErro, Vazio } from "@/components/Estados"
import { listarEventos } from "@/services/eventos"

export function Vitrine() {
  const { data: eventos, isPending, error, refetch } = useQuery({
    queryKey: ["eventos"],
    queryFn: listarEventos,
  })

  if (isPending) return <CarregandoCards />
  if (error) return <MensagemErro erro={error} tentarDeNovo={refetch} />

  return (
    <section className="grid gap-8">
      <header className="grid gap-2">
        <h1 className="text-4xl font-extrabold sm:text-5xl">Programação do StartUB 2026</h1>
        <p className="max-w-prose text-lg text-muted-foreground">
          Oficinas, minicursos e o hackathon do congresso. Escolha um evento e faça sua
inscrição.
        </p>
      </header>

      {eventos.length === 0 ? (
        <Vazio>A programação ainda não tem eventos.</Vazio>
      ) : (
        <div className="grid gap-3">
```

```
          {eventos.map((evento) => (
            <EventoCard key={evento.id} evento={evento} />
          ))}
        </div>
      )}
    </section>
  )
}
```

O `useQuery` recebe a **chave** e a **função** que busca. Devolve o dado e o estado: enquanto `isPending` for verdadeiro, desenhamos os esqueletos; se houver `error`, o alerta; depois disso, o TypeScript sabe que `eventos` existe, e o `map` não precisa de `?`.

PASSO 6.5 · O DETALHE COM DADOS REAIS

Atualize `src/pages/DetalheEvento.tsx`:

src/pages/DetalheEvento.tsx

```tsx
import { Link, useParams } from "react-router"
import { useQuery } from "@tanstack/react-query"
import { ArrowLeft } from "lucide-react"
import { DataDoEvento } from "@/components/DataDoEvento"
import { MensagemErro } from "@/components/Estados"
import { Badge } from "@/components/ui/badge"
import { Skeleton } from "@/components/ui/skeleton"
import { ehGratuito, formatarData, formatarValor } from "@/lib/formatar"
import { buscarEvento } from "@/services/eventos"

export function DetalheEvento() {
  const { id } = useParams()
  const eventoId = Number(id)

  const { data: evento, isPending, error } = useQuery({
    queryKey: ["eventos", eventoId],
    queryFn: () => buscarEvento(eventoId),
  })

  if (isPending) return <Skeleton className="h-96 rounded-xl" />
  if (error) return <MensagemErro erro={error} />

  return (
    <article className="grid gap-6">
      <Link to="/" className="flex w-fit items-center gap-1 text-sm text-muted-foreground
hover:text-foreground">
        <ArrowLeft className="size-4" /> Voltar para a programação
      </Link>

      <div className="flex items-start gap-6">
        <DataDoEvento data={evento.data} className="pt-1" />
        <div className="grid gap-3">
          <Badge variant={ehGratuito(evento.valor) ? "secondary" : "default"}>
            {formatarValor(evento.valor)}
          </Badge>
          <h1 className="text-4xl leading-none font-extrabold sm:text-5xl">{evento.titulo}
```

```
</h1>
        </div>
      </div>

      {evento.descricao && <p className="max-w-prose text-lg text-muted-foreground">
{evento.descricao}</p>}

      <dl className="grid gap-4 rounded-xl bg-muted p-5 sm:grid-cols-3">
        <div>
          <dt className="text-sm text-muted-foreground">Quando</dt>
          <dd className="font-semibold">{formatarData(evento.data)}</dd>
        </div>
        <div>
          <dt className="text-sm text-muted-foreground">Onde</dt>
          <dd className="font-semibold">{evento.local}</dd>
        </div>
        <div>
          <dt className="text-sm text-muted-foreground">Vagas</dt>
          <dd className="font-semibold">{evento.vagas}</dd>
        </div>
      </dl>
    </article>
  )
}
```

Agora apague o arquivo `src/dados-de-exemplo.ts`. Nenhum componente usa mais os dados fixos.

> **Atenção.** A chave precisa conter **tudo de que o dado depende**. Com `queryKey: ["evento"]`, sem o id, o cache guardaria um evento só: abrir o evento 2 depois do 1 mostraria o 1 por um instante. Com `["eventos", eventoId]`, cada evento tem sua entrada.

> **CONFIRA ANTES DE SEGUIR.** Logado, a vitrine mostra os eventos da API, com os esqueletos piscando por um instante antes. A aba Network mostra **uma** requisição a `/eventos`, com o cabeçalho `Authorization`. Voltar do detalhe para a vitrine não faz outra, porque o dado está fresco. Abra o devtools do TanStack Query (o botão redondo) e veja as chaves `["eventos"]` e `["eventos", 2]`. Por fim, troque à mão o token guardado no Local Storage por qualquer texto e recarregue: a API responde 401, a sessão é apagada e você volta para o login.

7. Fechamento



Figura 7. A vitrine ao fim do Dia 1, no celular

> **Fechamento.** Hoje você montou uma aplicação React com TypeScript que **conversa com uma API real e protegida**. Componentes recebem dados por props e não os alteram. O roteador escolhe a página pelo endereço sem recarregar o site, e a rota protegida só deixa passar quem entrou. O **token** identifica cada requisição, e um único lugar, a função `requisicao`, cuida dele. O **Context** guarda quem está logado. O `useState` guarda o que se digita. E o **TanStack Query** cuida do dado do servidor, com cache, carregando e erro.

No segundo encontro, a aplicação passa a **escrever** na API: busca e filtro, formulário de inscrição com validação, acompanhamento do pagamento, o painel de eventos e, por fim, o cadastro e o login refeitos com as mesmas ferramentas da inscrição.

Quem não terminou alguma etapa começa o segundo encontro pela branch `dia-1` do repositório do minicurso, que é o projeto exatamente como ele está ao fim deste material:

```
TERMINAL
> git switch dia-1
> npm install
```

Quem não usa Git recebe do professor o mesmo projeto num arquivo, o **Ponto de partida do Dia 2.zip**: basta descompactar, rodar `npm install` e `npm run dev`.

Para praticar em casa

São exercícios opcionais, que usam só o que foi visto hoje.

1. Mostre no cabeçalho o primeiro nome de quem está logado, como "Olá, Ana". Dica: `useAuth()`.
2. Na página do evento, mostre quanto falta para ele acontecer, como "em 12 dias". Use a função `paraDate` de `formatar.ts`.
3. No login, acrescente um botão que mostra e esconde a senha. É um `useState` com `true` ou `false` trocando o `type` do campo entre `password` e `text`.
4. Mostre acima dos cartões quantos eventos a API devolveu, como "2 eventos".

Desafio. Crie a página `/conta`, protegida, que busca `GET /user` com `useQuery` e mostra o nome, o e-mail e a data de criação da conta.

Referência rápida

Preciso de	Use
um pedaço de tela reutilizável	componente: função com nome em maiúscula que devolve JSX
passar dados para um componente	props, descritas por uma `interface`
repetir um elemento para cada item	`lista.map((item) => <X key={item.id} ... />)`
mostrar algo só às vezes	`{condicao && <X />}` ou `condicao ? <A /> : <B />`
navegar sem recarregar	`<Link to="/eventos/2">`
ler o id do endereço	`const { id } = useParams()`
buscar dado da API	`useQuery({ queryKey: [...], queryFn: ... })`
guardar o que existe só na tela	`const [valor, setValor] = useState(inicial)`
saber quem está logado	`const { usuario } = useAuth()`
mandar para outra página ao desenhar	`<Navigate to="/entrar" replace />`

Comando	Para que serve
`npm run dev`	servidor de desenvolvimento

Comando	Para que serve
`npm run lint`	procura erros e más práticas no código
`npm run build`	gera a versão de produção na pasta `dist`
`npx shadcn@4.23.0 add <nome>`	acrescenta um componente do shadcn